

Étude d'un décodeur de parité croisée

I. Présentation :

Il peut être intéressant, lors de l'envoi de données sur une liaison où des erreurs de transmission peuvent survenir, de coder l'information pour détecter et éventuellement corriger les bits erronés. Le codage a pour but d'amener un certain degré de redondance, au prix d'un débit utile diminué, afin qu'un décodeur en bout de liaison ait assez d'informations pour décider de la validité ou non des bits reçus.

Un des codages les plus simples consiste à ajouter un bit de parité à un groupe de bits de données. À la réception, on calcule à nouveau un bit de parité sur les bits de données reçus dont la comparaison avec celui calculé à l'émission permet de détecter une erreur. Cette simplicité se traduit par une efficacité médiocre dans le sens où, d'une part, une erreur détectée ne peut conduire à aucune correction et où, d'autre part, deux erreurs, lors de la transmission, annulent l'écart de parité et ne sont donc pas détectées.

Afin d'améliorer les performances du codage de l'information, on préfère traiter un ensemble de données sous la forme d'un bloc (matrice) constitué par l lignes et m colonnes. On ajoute alors un bit de parité à la fois à chacune des lignes et à chacune des colonnes. Le dernier bit de la matrice est utilisé pour calculer la parité des bits de parités des lignes et des colonnes. La figure suivante montre comment le codage est réalisé :

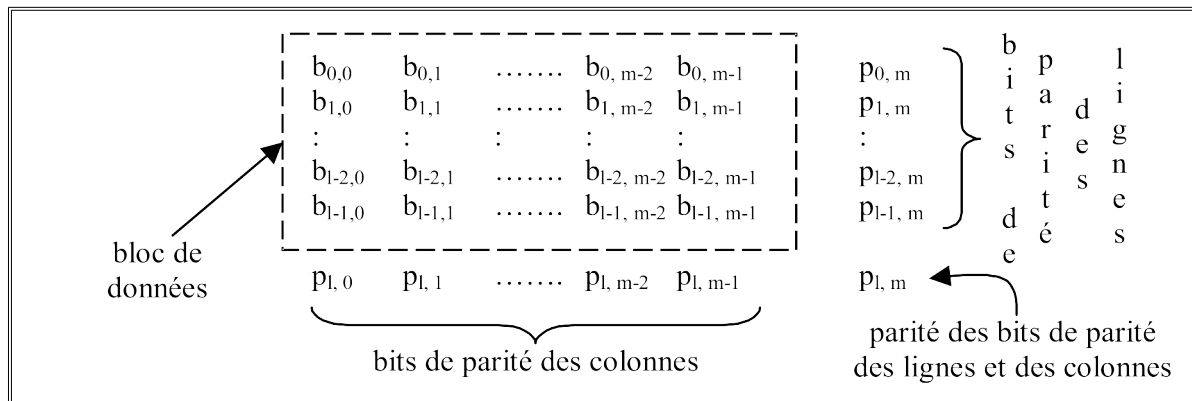


Figure 1 : Ajouts des bits de parité des lignes et des colonnes

On appellera la suite de bits $\{p_{0,m}, p_{1,m}, \dots, p_{l-2,m}, p_{l-1,m}\}$ vecteur parité lignes, $\mathbf{Vpl}$, et la suite de bits $\{p_{l,0}, p_{l,1}, \dots, p_{l,m-2}, p_{l,m-1}\}$ vecteur parité colonne, $\mathbf{Vpc}$. On distinguera les bits calculés par le décodeur de ceux reçus en ajoutant à ces derniers l'extension $_r$.

II. Cas à considérer :

Une erreur est survenue lors de la transmission :

Un bloc, comprenant données et parités, est transmis au récepteur avec une erreur. On a alors le cas présenté sur la Figure 2. Les bits de parités calculés à la réception diffèrent de ceux transmis. Cette différence peut être mise à profit : en effet, on constate que le bit erroné a une position dans le bloc correspondant à l'intersection ligne – colonne des bits de parité calculés différents de ceux reçus. On peut dans ce cas précis, non seulement détecter l'erreur, mais également la corriger.

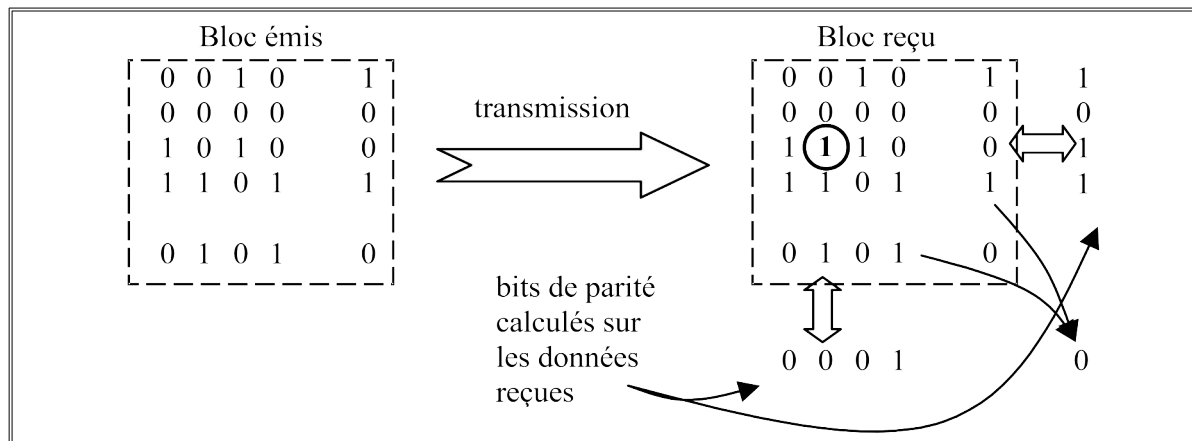


Figure 2 : Une erreur de transmission est survenue dans le bloc de données

L'erreur peut également entacher non pas un bit du bloc de données mais un bit de parité. Si l'on reprend l'exemple précédent, on obtient alors le cas présenté sur la Figure 3. On constate alors que les bits de parité des lignes, reçus et calculés, sont identiques mais pas ceux des colonnes. L'erreur n'est pas alors localisée dans le bloc de données mais seulement au niveau d'un bit de parité. On peut alors la corriger, en réalité simplement ne pas tenir compte du bit incriminé, puisque de toute façon celui-ci sera ignoré dans la suite du traitement des bits de données par le système. On peut remarquer que dans ce cas le bit de parité globale, calculé sur les bits de parités reçus, n'est plus correct.

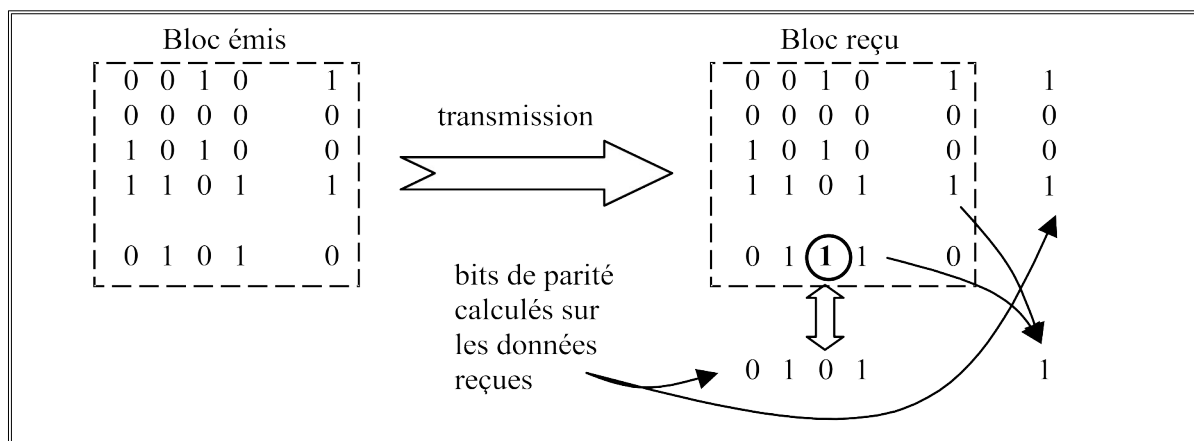


Figure 3 : Une erreur de transmission est survenue sur l'un des bits de parité

Deux erreurs ou plus sont survenues lors de la transmission :

Bien évidemment, le codage envisagé, même s'il est plus performant qu'une simple parité, présente des limites à la détection et à la correction d'erreurs. Par exemple, si deux erreurs surviennent lors de la transmission, on pourra éventuellement les détecter mais pas les corriger. Quand deux erreurs surviennent au cours de la transmission d'un bloc deux cas sont à considérer : soit la combinaison produite est identifiable comme étant fausse, soit elle est prise pour une configuration correcte. Si on la reconnaît comme fausse, le système demandera un nouvel envoi des données ; si ce n'est pas le cas, le codage est tenu en échec et des données fausses, ou corrigées à tort, seront fournies au système. Cette situation n'est pas forcément rédhibitoire : en effet, si la probabilité de mal transmettre un bit est " p " celle de mal en transmettre deux est " p^2 ". Si " p " est raisonnablement faible, les situations mettant le codage en défaut sont très peu probables ; on peut donc en accepter le risque. Si ce risque apparaît trop

grand vis-à-vis d'une application donnée, il faudra alors envisager un codage plus puissant, vraisemblablement au prix d'une complexité accrue et/ou d'un débit utile inférieur.

Résumé des cas :

Il n'y a pas d'erreur de transmission ou leur combinaison induit la même conclusion :

- V_{pl} et V_{pc} sont identiques respectivement à V_{pl_r} et V_{pc_r} et $p_{l,m}$ est identique à p_{l,m_r} : le décodeur transmet les bits de données reçus au système.

Une seule erreur peut être corrigée et correspond aux cas suivants :

- V_{pl} et V_{pc} diffèrent d'un seul bit respectivement de V_{pl_r} et V_{pc_r} , et $p_{l,m}$ est identique à p_{l,m_r} : le décodeur corrige le bit erroné et transmet les bits de données au système.
- la suite $\{V_{pl}, V_{pc}\}$ diffère de 0 ou 1 bit au plus de la suite $\{V_{pl_r}, V_{pc_r}\}$ et $p_{l,m}$ est différent de p_{l,m_r} : le décodeur transmet les bits de données reçus au système.

Dans tous les autres cas une erreur est détectée sans que l'on ait les moyens de la corriger : le décodeur indique au système que les données sont entachées d'erreurs.

III. Signaux d'interfaçage du décodeur de parité croisée :

Le but du travail demandé ici est l'étude et la conception d'un décodeur fonctionnant sur le principe de la parité croisée. On travaillera sur des blocs de données de 7 bits par 7 bits (m et l égaux à 7). Le décodeur sera intégré au sein d'un système numérique élaboré autour d'une interface d'interconnexion Avalon. La vue externe du décodeur est présentée sur la Figure 4.

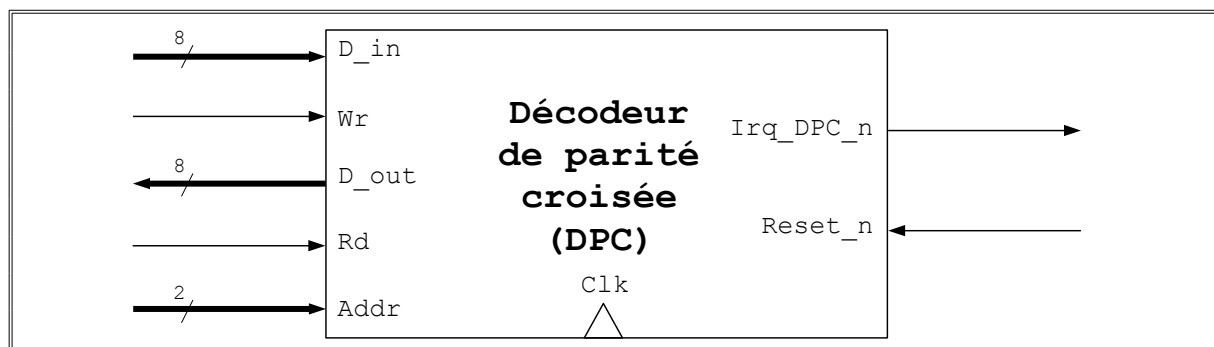


Figure 4 : Vue externe du décodeur DPC

La fonction des signaux du décodeur DPC est la suivante :

- **D_in** : bus de données 8 bits en écriture,
- **Wr** : validation d'un transfert de données en écriture,
- **D_out** : bus de données 8 bits en lecture,
- **Rd** : validation d'un transfert de données en lecture,
- **Addr** : bus d'adresse du périphérique,
- **Reset_n** : initialisation asynchrone du périphérique, actif à l'état bas,
- **Clk** : horloge de fréquence $f_{Clk} = 25$ MHz,
- **Irq_DPC_n** : demande d'interruption, active à l'état bas.

IV. Interface du décodeur DPC :

La communication avec le décodeur DPC est assurée par l'intermédiaire des registres présentés sur la Figure 5. Ces registres ont un format de 8 bits.

Registre	Offset	Accès	Format						
			MSB						LSB
DPCStatus	0	R	Error	Corr	Done	-	-	-	-
DPCControl	1	R/W	-	-	IrqEn	-	-	-	Decod
DPCDataIn	2	W	Data in (8 bits)						
DPCDataOut	2	R	Data out (8 bits)						

Figure 5 : Registres du décodeur DPC

Rôle des registres :

- **DPCDataIn, DPCDataOut** : le décodeur DPC stocke les données à décoder dans une pile de type FIFO. Un composant générique est fourni pour implémenter la pile (voir fiche dédiée "Pile FIFO").
- **DPCControl** : ce registre comporte 2 bits permettant le contrôle du décodeur.
 - Decod : lorsqu'il est mis à '1', le décodeur DPC commence le décodage de toutes les données qui ont été stockées dans la FIFO. Ce bit est automatiquement remis à 0 par le décodeur lorsque le décodage est terminée,
 - IrqEn : sert de masque des interruptions. Lorsqu'il est à '0' le décodeur ne peut pas demander d'interruption (Irq_DPC_n est toujours inactif).
- **DPCStatus** : ce registre comporte bits suivants donnant l'état du périphérique
 - Done : indique que le traitement est terminé. Ce bit est actif lorsque toutes les données placées dans la pile ont été décodées. Lorsque les interruptions sont autorisées, il génère une demande d'interruption auprès du processeur (Irq_DPC_n est alors actif). Ce bit est automatiquement remis à 0 lorsque le registre est lu,
 - Corr : indique que les données décodées ont été corrigées et ne comportent plus d'erreurs connues,
 - Error : les données décodées comportent des erreurs ne pouvant être corrigées.

V. Utilisation du décodeur de parité croisée :

Lorsque le processeur souhaite faire décoder des données, il les place dans la pile FIFO interne du décodeur en écrivant dans le registre **DPCDataIn**. Les données, transmises via le bus d'entrée **D_in** au format 8 bits, représentent la matrice à décoder. Le décodage peut commencer lorsque le bit Decod est activé. Après décodage, chacune des données constituant la matrice est stockée dans la pile interne du décodeur. Le processeur récupérera l'ensemble des données présentes dans la pile, par lecture du registre **DPCDataOut**.

L'accès logiciel au composant DPC sera effectué par l'intermédiaire d'une API à définir et à concevoir permettant, au minimum, l'accès aux registres du coupleur. Le programme à développer permettra de transmettre une matrice au décodeur et demandera le décodage de ces données. Après décodage, le programme lira et affichera les données sur un terminal.